devops engineer - easy guide

DevOps Engineer — Easy Guide

For someone new to the role. No container/cloud background assumed.

Working assets: `infra/`, `docker-compose.yml`, `backend/Dockerfile`, `tasks.py`, `.github/workflows/ci.yml`

---

1. What “DevOps” means for this project

Someone else builds the forecasting model and the API that serves it. Your job is different: get that code running reliably on *someone else’s machine* — a teammate’s laptop, a cloud VM, a CI runner — not just yours. That covers four things:

- **Build** — turn source code into something runnable (a container image).
- **Ship** — get that runnable thing to where it needs to run.
- **Run** — start it correctly, with the right configuration and data.
- **Debug** — when it doesn’t work, figure out why, fast, from symptoms.

If you read exactly one other file after this one, read `infra/docs/onboarding.md` — it’s the step-by-step, hands-on version of everything explained conceptually here.

2. The one thing that will bite you first

This API needs a **130 MB folder of pre-built data files** — a database and two data sidecar files — that is **not stored in git**. It has to be generated locally, once, before anything works:

```
python tasks.py build-db
```

Without it, the API process actually starts fine — it just can't answer any real question. If your orchestration is set up correctly, this shows up as "the frontend never starts" (because it's waiting for the API to report *ready*, not just *alive* — see §4). If it's set up wrong, it looks exactly like a mysterious hang, and debugging it as a hang wastes hours. It's the single most common "why is this broken" moment on this project — internalise it now.

---

3. Images vs. containers — the recipe and the cake

Two words get used interchangeably by beginners and that causes confusion:

- A **Docker image** is a recipe — a static, layered description of "install Python, copy this code, set this command to run." It doesn't do anything by itself.
- A **container** is what you get when you actually run an image — a live, running process with its own filesystem view, isolated from the host.

You can build one image and start many containers from it (that's how you scale — more identical copies, not bigger ones). This project's backend Dockerfile actually defines **two different recipes from one file**, called *targets*:

Target	Think of it as	Needs the research data?
`api`	A lean backpack — just the API and a database reader	No
`full`	The same backpack plus a whole toolbox — adds the machine-learning libraries so it can re-run the model live	Yes

`api` is the default, and it's what actually runs in the deployed instance (§8).

Nobody had actually built these images for a long time, and it showed. `backend/Dockerfile` had `pip install --require-hashes=false` — but `--require-hashes` is a flag with no value, like a light switch, not a dial. Writing `=false` on it is invalid syntax, and pip refuses to run at all. Nobody caught it because the image had never actually been built — the machine it was written on didn't have Docker installed. The lesson

generalises, and this project's own history proves it more than once (§8 has two more examples): **a Dockerfile, a CI workflow, or a cloud config that has never actually been run is a Dockerfile you don't know works, no matter how carefully it was written.**

---

4. "Alive" vs. "Ready" — the probe distinction that matters most

This is the single most important concept to get right in any containerised system, and this project has a very clean example of getting it wrong being disguised as something else entirely.

- **Liveness** answers: *is the process still running?* (`/api/v1/health`)
- **Readiness** answers: *can it actually do its job right now?* (`/api/v1/ready`)

A process can be perfectly *alive* — the Python interpreter is running, it answers HTTP requests — while being completely unable to serve a real answer, because the data file it needs isn't there yet. If your readiness check is wired to the wrong endpoint, an orchestrator will happily send real traffic to a container that has nothing to say, and callers get errors that look random and are actually completely deterministic.

The rule: point your orchestrator's readiness probe at `/ready`, not `/health`. This project's frontend container is configured to wait for the API's `service_healthy` condition — which is *why* a missing data file makes the whole stack appear to hang rather than just serve broken responses.

---

5. Environment variables and secrets — labelled boxes vs. a locked envelope

Almost everything configurable about this system is an **environment variable** — a labelled setting you can change without touching code. Think of each one as a labelled box: `NPN_LOG_LEVEL` says how chatty the logs are, `NPN_DATA_DIR` says where to find the data files, and so on. Most have sensible defaults.

One of them is different: `GEMINI_API_KEY`, which unlocks the AI assistant feature. That one isn't a labelled box — it's a locked envelope. It:

- is optional — leave it out and the assistant reports "unavailable" while every other feature works normally;
- is never written into the container image itself (so anyone who gets a copy of the image cannot extract it);
- is only ever injected at the moment a container *starts*, from a `.env` file, or — in the real cloud deployment (§8) — a secret store;
- never appears in a log, a response, or the browser's JavaScript bundle.

Docker Compose is not a secret store, and neither is a `.env` file checked into anything. Locally, a `.env` beside `docker-compose.yml` is fine. In the real AWS deployment, the key instead lives in **AWS Systems**

Manager Parameter Store, encrypted, and is fetched by the deploy machinery at the moment the container starts — never typed into a GitHub secret, never baked into an image. Same idea as local dev, one level more careful.

---

6. Docker Compose — conducting more than one container together

A `docker-compose.yml` file describes a small *system* of containers that need to run together and talk to each other — here, an API container and a frontend container. It handles:

- **Networking** — the containers get their own private network and can reach each other by name (the frontend talks to `api:8000`, never `localhost:8000` — that would mean "myself").
- **Startup order** — "don't start the frontend until the API is healthy."
- **Volumes** — mounting the 130 MB data folder into the API container, **read-only**, so the container can read it but never accidentally corrupt it.
- **Resource limits, healthchecks, restart policy** — all declared once, in one file, instead of remembered as a list of manual `docker run` flags.

This project has **three different flavours** of the compose file, and the difference matters:

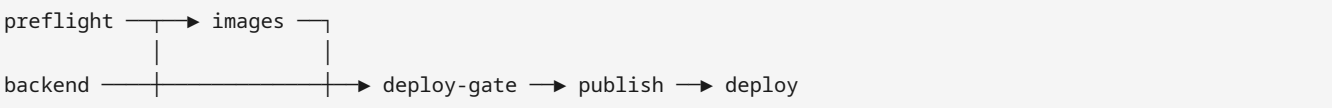
File	Where it runs	How it gets its images
`docker-compose.yml` (+ prod overlay)	Your laptop, a single dev host	**Builds** the images itself, locally
`docker-compose.deploy.yml`	The real deployed EC2 box	**Pulls** already-built images from a registry
`docker-compose.inference.yml`	Either, opt-in	Switches the API build target to `full`

The deployed box never has the source code, a Dockerfile, or a build toolchain on it at all — it just runs whatever image a build pipeline handed it. That split — "build here, run there" — is the core idea behind almost every real production deployment, and this project now has a small, real example of it (§8).

---

7. CI — a relay race that catches mistakes before they ship

CI (Continuous Integration) means: every time someone proposes a code change, a machine automatically runs a series of checks — build it, boot it, lint it — *before* a human has to trust it. Think of it as a relay race, each runner only starting once the previous one succeeds:



```
                │              │
frontend ──────┴──────────────┘
```

The machine that runs CI has no copy of the 130 MB data file (§2) and never will — it's not in git, on purpose. So CI can prove the *code* is correct and the *containers boot correctly with no data*, but it structurally cannot prove the deployed system serves the *right numbers*. That's what the canary (§10) and the final deploy step exist for.

A note on what "backend" and "frontend" actually check right now. At an earlier point this project had a real automated test suite — 149 backend tests, 65 frontend tests, run on every CI push. That suite has since been removed from the repository entirely, along with the CI steps that ran it. What CI verifies today is narrower: the app *imports* cleanly, the committed API schema is current, the frontend *typechecks* and *builds*, and the containers *boot* and *degrade correctly* with no data. That is real signal, but it is a different, smaller kind of signal than "the logic is correct" — worth knowing plainly rather than assuming a green CI run still means what it used to.

This project used to stop at `deploy-gate` on purpose — CI proved the code was good but didn't deploy anywhere, because there was nowhere to deploy *to*. Once there was a real target (§8), two more runners were added to the end of the relay: `publish` (build the real images, push them somewhere) and `deploy` (tell the real box to update). Adding CD only makes sense once a target actually exists — a "deploy" step pointing at nothing is worse than no deploy step, because it looks like a capability nobody actually has.

---

8. Shipping it for real — EC2, GitHub Actions, and "push-style GitOps"

This is genuinely new ground for the project, so it's worth walking through slowly, because every piece maps to a concept you'll meet again on any cloud deployment — and because getting it running for real, rather than just writing it, is exactly what found the bugs in §8.1.

The goal: every time code is pushed to `main` and passes every CI check, a real server automatically updates itself to run the new version — with no human typing a deploy command, and with nothing sensitive (an AWS password, a database key) ever pasted into a chat window or stored as plain text anywhere.

How the pieces fit together:

1. **A real, always-on server exists** — one EC2 instance, a small rented Linux computer. It runs Docker and nothing else interesting; it doesn't even have this project's source code checked out on it. It gets a permanent public address (an "Elastic IP" — a public IP that doesn't change even if the server restarts, unlike the default kind).
2. **The server has no SSH access at all.** Normally you'd log into a server with a password or an SSH key. This one has **no open port for that** — instead it uses **AWS Systems Manager (SSM)**, which lets AWS itself run commands on the box on your behalf, authenticated entirely by AWS permissions rather than a key that could be lost, stolen, or left in a config file. No port to attack, nothing to leak.

3. **GitHub Actions proves who it is without a stored password.** The old way to let a CI pipeline talk to AWS was to generate a long-lived AWS access key and paste it into GitHub as a secret — exactly the kind of thing that leaks in a repo history or a misconfigured log line. Instead, this project uses **OIDC** (OpenID Connect): GitHub's own servers hand the pipeline a short-lived, cryptographically signed token proving "this is genuinely a run of this repo, on the `main` branch, right now" — and a role in AWS is configured to trust *only* tokens with exactly that shape. There is no password to leak, because there is no password.

4. **Images are built once, in the pipeline, and pushed to a registry.** `publish` builds the real API and frontend images (not just boot-tests them, the way the earlier `images` CI job does) and pushes them to **ECR** (AWS's Docker registry), tagged with the exact commit hash — never `latest`, because "latest" makes it ambiguous what's actually running and impossible to cleanly roll back to.

5. **The `deploy` job tells the server what to do, and waits.** It uses SSM to run a small script *on the box*: log in to the registry, fetch the secret key from AWS's secret store, and run `docker compose up -d` with the new image tags. The pipeline then polls until that command reports success — or fails loudly if it doesn't, rather than reporting "deployed" optimistically.

6. **The very last step is the same canary check from §10**, run from GitHub's own servers against the box's real public address. A deploy that returns `200 OK` on every endpoint but serves the wrong numbers is not a successful deploy, and this step is what tells the difference.

Why this whole shape is called "push-style GitOps": the trigger is a `git push`, not a human clicking "deploy," and the desired state (which image tag should be running) is decided entirely by what's in git. It's not *true* GitOps in the strict Kubernetes sense (where a separate agent continuously reconciles the running state against git on its own schedule) — for one box running Docker Compose, that machinery would be solving a problem this project doesn't have. This is the honest, right-sized version of the same idea.

The one thing that doesn't travel with a normal `git push`: the 130 MB data layer. It isn't generated by CI (CI has no access to the research artefacts it's built from) and it isn't small enough to embed in a deploy script. It was shipped to the server **once**, through a private cloud storage bucket, as a one-time bootstrap step — not something that happens on every deploy.

8.1 Four real bugs, found only by actually running this for real

Every one of these passed a human reading the code carefully. None of them were caught until the pipeline actually ran against real AWS resources for the first time — which is exactly the lesson from §3, playing out again at a bigger scale.

1. **A stale copy-pasted security fingerprint.** The one-time setup script hard-coded a "thumbprint" for GitHub's identity-token service — a short fingerprint of the TLS certificate AWS uses to prove it's really talking to GitHub. That value was correct once, but GitHub's certificate provider changed at some point, and the old fingerprint silently stopped matching. The error this produces (`Not authorized to perform sts:AssumeRoleWithWebIdentity`) gives no hint that the *fingerprint* is the problem — it

looks identical to a permissions mistake. The fix: compute the fingerprint live from the real certificate every time the setup script runs, instead of trusting a value someone typed in once.

2. **An identity check that didn't match reality.** AWS was told to trust tokens shaped like `repo:owner/repo:ref:refs/heads/main`. That's the textbook format documented everywhere — but the *actual* token GitHub sent for this repository was shaped like `repo:owner@12345/repo@67890:ref:refs/heads/main`, with extra numeric IDs baked in (GitHub's own protection against someone renaming an account to hijack an old trust relationship). The fix wasn't found by reading documentation harder — it was found by reading AWS's own audit log (CloudTrail), which records the *exact* identity string a rejected request actually presented.

3. **A safety setting that broke the container instead of protecting it.** The frontend container runs with a read-only filesystem — a real hardening measure, not decoration. But nginx's own startup process needs to write one specific config file at boot, and that one path had no writable exception carved out for it. The container didn't run *unsafely* — it didn't run **at all**, crash-looping immediately. The fix was one extra line granting exactly that one path a small amount of temporary, in-memory writable space — the same pattern already used for nginx's two or three other startup scratch paths, just missing one.

4. **A security header that silently vanished.** The site was *supposed* to send three standard security headers on every page. It was — but only on some responses. One web-server configuration rule turned out to mean "if a more specific rule sets its own version of a setting, it silently replaces the general one, it doesn't add to it" — and two specific rules (for the homepage and for cached assets) each set one unrelated setting of their own, which silently dropped the three security headers for almost every real page load. The fix was mechanical once found — repeat the three headers in both specific rules — but finding it required actually inspecting real response headers from a real running site, not reading the config and reasoning about what it *should* do.

The common thread across all four: **reading code carefully is necessary, but it is not the same thing as verifying code, and it will never catch everything that only shows up when the code actually runs.**

---

9. Observability — how you know what's happening inside

A running container is a black box unless you deliberately make it tell you things. "Observability" is the umbrella word for the three ways this project lets you see inside it: **logs** (what happened), **health signals** (is it working right now), and **request tracing** (which specific request caused that one log line).

Logs are structured, not free text. Every log line the API writes comes out as a single-line JSON object — `{"time": ..., "level": ..., "logger": ..., "msg": ...}` — instead of a human-readable sentence. That's a deliberate trade: it's slightly less pleasant to read with your own eyes in a terminal, but it means a log-aggregation tool (if you ever add one) can parse every line reliably without guessing at a text format. `NPN_LOG_LEVEL` controls how chatty it is — `INFO` normally, `DEBUG` only while actively diagnosing something, never left on, because it's noisy.

Every request gets an ID, and it follows the request everywhere. The API stamps an `X-Request-ID` on every response (reusing one you send in, or generating one if you didn't). If a request errors, that same ID appears both in the error response *and* in the server log line describing what went wrong — so "the user reported error ID `a1b2c3d4e5f6`" is enough to find the exact log entry, with the full detail, without leaking that detail to the person who hit the error. Every response also carries `X-Response-Time-ms`, and anything slower than one second gets an automatic "slow request" log line — you don't have to go looking for slow requests, they flag themselves.

Readiness isn't just yes/no — it can say "degraded." `/api/v1/ready` doesn't only report `ready: true/false`. It can also report `degraded: true` — meaning the core forecast data is fine, but one of the optional sidecar files (history or backtest) isn't loading correctly. That's a genuinely useful middle state: "still serving the main thing correctly, but something secondary needs attention" is different from either "fully healthy" or "down," and collapsing that distinction into a plain boolean would throw away information an operator actually wants.

What's honestly missing: there's no metrics endpoint (nothing a tool like Prometheus can scrape) and no centralised place logs get shipped to — right now, "look at the logs" means either `docker compose logs` on the deployed box (over SSM, since there's no SSH), or `aws ssm send-command` to fetch them. That's a real limitation for a long-running, multi-replica production deployment, and a complete non-issue for a single-host demo. If you're the one asked to close that gap, the shape of the fix is: keep emitting the same structured JSON logs (don't reinvent the format), point them at a log shipper instead of stdout, and add a `/metrics` endpoint alongside the existing `/health` and `/ready` ones rather than replacing them — liveness and readiness answer a different question than a metrics dashboard does, and you want all three.

---

10. The one number that proves everything is actually working

The frozen forecast for one specific product (`CA_3 / FOODS_3_090`) totals exactly **3331.3681** over its 28-day forecast window. That number never changes, because the model is frozen and the data behind it never changes either.

This project uses that fact as a **canary** — a single, cheap, unambiguous check that answers "is this deployment actually serving the right, validated data?" rather than just "does it respond to HTTP requests at all." A container can answer every request with a `200 OK` and still be silently serving stale or half-built data — the canary is what tells those two situations apart. It's checked twice now: once by hand with `python tasks.py smoke`, and once automatically as the very last step of every deploy (§8).

Never consider a deployment finished until this passes.

---

11. Pausing a cloud deployment without losing anything

A cloud server costs money for every hour it runs, whether or not anyone is using it. When you're not actively demoing or testing, the right move is to **stop the instance**, not delete anything:

- `aws ec2 stop-instances` — the box shuts down, compute billing stops, and the site goes offline. The disk (with your data already on it), the permanent IP address, the container registry images, the IAM roles, and the stored secret all stay exactly as they were.
- `aws ec2 start-instances` — the box comes back with everything still in place. Docker, the data layer, and the last-deployed containers are all still there; you may just need to `docker compose up -d` again if the containers themselves weren't set to restart automatically.

This is the cloud equivalent of closing your laptop lid rather than wiping the drive — cheap, safe, and completely reversible. It's also worth pairing with a habit from CI: if you push code while the box is stopped, the `deploy` job will run and fail (it can't reach a stopped instance) — that's expected and harmless, not a sign anything is broken. The instance itself is never touched by a failed deploy attempt.

---

12. The habit worth building: symptom → cause → fix, not guessing

When something's broken, resist the urge to start changing things randomly. This project's troubleshooting doc is organised as *symptom → cause → fix*, ordered by how often each actually happens — and the top three cover the large majority of real incidents you'll hit:

1. **"It just hangs."** Almost always the missing data file from §2, not an actual hang. Confirm with `curl localhost:8000/api/v1/ready`.
2. **"The Docker build is sending gigabytes."** A new top-level folder was added to the repo and nobody updated the ignore-list, so everything in it gets uploaded to the build. Fixable in one line, once you know to look.
3. **"It answers, but the numbers are wrong."** Rebuild the data file — never assume it's fine just because the server responds.

A fourth, learned directly and repeatedly from this project's own history (§3, §8.1): **"it builds on my machine," or "the code looks right," proves nothing if it has never actually been run for real.** The fix isn't "review more carefully" — it's "make sure it actually runs somewhere, regularly, against the real thing," which is exactly what wiring CI to really build the images, and actually running the AWS deploy path end to end, now guarantees.

Always run these three commands first, in this order, before doing anything else:

```
python tasks.py preflight     # is the configuration sane, before you build?
python tasks.py docker-ps     # what is actually running right now?
python tasks.py smoke         # what specifically is broken?
```

`smoke` names the failing piece by design — read its output before guessing.

---

13. Where to go next

- `infra/docs/onboarding.md` — hands-on, step by step, ~30 minutes from a clean clone to a verified running stack.
- `infra/docs/troubleshooting.md` — the full symptom → cause → fix reference.
- `infra/scripts/aws_bootstrap.sh` — the one-time script that provisions everything described in §8: ECR, the OIDC role, the EC2 instance role, the security group, the data bucket.
- `detailed-guide.md` — the same concepts, tied to exact files, resource names, and the real incidents found while building this.